

# Banking Case Study 3

## Failed Payment Reversal and Transaction Recovery Testing in a Digital Banking Platform

### Case Study Summary

This case study demonstrates how I would test failed payment handling in a digital banking platform. The focus is on validating what happens when a customer initiates a payment, the money is deducted or reserved, but the payment fails before the recipient receives the funds.

The objective is to confirm that the system handles the failure correctly by updating the transaction status, reversing the debit where required, restoring the customer balance, creating a complete audit trail and ensuring that all records remain consistent across the customer view, statement, admin portal, API response and database.

This case study connects strongly to my regulated iGaming testing experience because I already work with financial logic, balances, payouts, transactions and compliance focused validation.

This case study uses fictional banking data only.

### 1. Business Problem

In banking, not every payment succeeds. A payment can fail because of network issues, recipient account problems, timeouts, system errors, duplicate detection, daily limit rules or insufficient funds.

The main risk is not only that the payment fails. The bigger risk is that the payment fails after money has already been deducted or reserved.

When this happens, the system must either leave the account untouched, reverse the debit, clearly show a pending state, restore the balance after reversal, provide a clear failure reason or prevent the transaction from being left in an unclear state.

| Failure Reason          | Example                                         |
|-------------------------|-------------------------------------------------|
| Network failure         | Connection drops during processing              |
| Recipient account issue | Recipient account is closed or invalid          |
| Timeout                 | Bank does not receive confirmation in time      |
| System error            | Payment service fails after debit               |
| Duplicate detection     | System blocks a repeated request                |
| Daily limit exceeded    | Customer exceeds allowed transaction limit      |
| Insufficient funds      | Balance becomes unavailable before confirmation |

| Situation                  | Expected System Behaviour                          |
|----------------------------|----------------------------------------------------|
| Payment fails before debit | No money must leave the account                    |
| Payment fails after debit  | Money must be reversed                             |
| Payment is pending         | Balance and status must clearly show pending state |
| Payment is reversed        | Customer balance must be restored                  |
| Payment is rejected        | Customer must receive a clear failure reason       |
| System timeout occurs      | Transaction must not be left in an unclear state   |

## 2. Why This Case Study Fits My Background

My current testing background involves validating regulated transactional systems where financial values must be accurate and traceable.

| iGaming Testing Area               | Banking Equivalent              |
|------------------------------------|---------------------------------|
| Game round interrupted after wager | Payment interrupted after debit |
| Player balance recovery            | Customer balance restoration    |
| Game history validation            | Payment history validation      |
| Back office record validation      | Banking admin portal validation |
| Transaction status validation      | Payment status validation       |
| Database transaction check         | Core banking database check     |
| Certification evidence             | Audit and compliance evidence   |

The banking scenario is different, but the testing principle is similar: when a transaction does not complete successfully, the system must still protect the customer money and maintain a reliable audit trail.

## 3. Business Risk

| Risk Area                          | Possible Impact                                       |
|------------------------------------|-------------------------------------------------------|
| Failed payment still deducts funds | Customer loses available balance incorrectly          |
| Reversal not created               | Customer funds are not restored                       |
| Incorrect transaction status       | Failed payment appears as completed                   |
| Missing audit trail                | Support team cannot investigate the issue             |
| Statement mismatch                 | Customer statement does not reflect the reversal      |
| API mismatch                       | External systems receive incorrect transaction status |
| Duplicate reversal                 | Customer is credited more than once                   |
| Delayed reversal                   | Customer cannot access funds on time                  |
| Reconciliation failure             | Finance team cannot balance records                   |
| Compliance risk                    | Bank may fail audit expectations                      |

## 4. Test Objective

The objective of this case study is to validate that a failed payment is handled correctly from initiation to final resolution.

| Validation Area      | Objective                                         |
|----------------------|---------------------------------------------------|
| Opening balance      | Confirm the customer balance before payment       |
| Payment attempt      | Confirm the payment request is created            |
| Debit or reservation | Confirm whether money is deducted or held         |
| Failure trigger      | Confirm that the payment failure is processed     |
| Reversal logic       | Confirm that money is restored if deducted        |
| Final balance        | Confirm that the customer balance is correct      |
| Transaction status   | Confirm that the status reflects the true outcome |

|                     |                                                              |
|---------------------|--------------------------------------------------------------|
| Transaction history | Confirm that failed and reversed records display correctly   |
| Statement view      | Confirm that statement entries are accurate                  |
| Admin portal        | Confirm internal records show the full transaction journey   |
| API response        | Confirm API returns the correct final status                 |
| Database record     | Confirm backend records are accurate and traceable           |
| Audit trail         | Confirm timestamps, references and status changes are stored |

## 5. Test Scope

### In Scope

| Area                           | Description                                                |
|--------------------------------|------------------------------------------------------------|
| Customer login                 | Confirm that the customer can access the banking platform  |
| Payment initiation             | Confirm that payment details can be captured               |
| Failure simulation             | Simulate or trigger a failed payment scenario              |
| Debit validation               | Confirm whether money is deducted or reserved              |
| Reversal validation            | Confirm whether funds are restored correctly               |
| Balance validation             | Confirm opening, temporary and final balance values        |
| Transaction status validation  | Confirm failed and reversed statuses                       |
| Transaction history validation | Confirm customer facing records are clear                  |
| Statement validation           | Confirm statement entries reflect the failure and reversal |
| Admin portal validation        | Confirm internal records show full payment journey         |
| API validation                 | Confirm API status matches the real transaction state      |
| SQL validation                 | Confirm database records match expected values             |
| Reconciliation                 | Compare customer view, statement, admin, API and database  |
| Defect reporting               | Log any mismatch with evidence and risk                    |

### Out of Scope

| Area                            | Reason                                       |
|---------------------------------|----------------------------------------------|
| Real banking systems            | This is a fictional portfolio case study     |
| Real customer accounts          | No real customer data is used                |
| Production payment rails        | Interbank settlement is not tested           |
| Fraud engine rules              | Fraud detection is not the focus             |
| Security penetration testing    | Security testing is outside this case study  |
| Legal compliance interpretation | The focus is QA validation, not legal advice |

## 6. Test Environment

| Item             | Example Value                 |
|------------------|-------------------------------|
| Application type | Demo digital banking platform |
| Platform         | Web banking or mobile banking |
| Environment      | QA environment                |

|                  |                                                                   |
|------------------|-------------------------------------------------------------------|
| Customer account | Fictional funded account                                          |
| Currency         | ZAR                                                               |
| Payment type     | EFT or account payment                                            |
| Failure type     | Payment service timeout                                           |
| API tool         | Postman                                                           |
| Database         | PostgreSQL or MySQL                                               |
| Defect tracking  | JIRA style defect tracking                                        |
| Evidence         | Screenshots, API responses, SQL results and reconciliation tables |

## 7. Test Data

| Field                                 | Value                   |
|---------------------------------------|-------------------------|
| Customer ID                           | CUST3001                |
| Sender account number                 | ACC3001                 |
| Recipient account number              | ACC4001                 |
| Opening balance                       | R6 000.00               |
| Payment amount                        | R1 250.00               |
| Transaction fee                       | R7.00                   |
| Temporary balance after debit         | R4 743.00               |
| Expected final balance after reversal | R6 000.00               |
| Payment reference                     | EFT9001                 |
| Failure reason                        | Payment service timeout |
| Final transaction status              | Reversed                |
| Currency                              | ZAR                     |

## 8. Failed Payment Calculation Logic

### Temporary Debit Calculation

If the system deducts the amount before failure:

Temporary Balance = Opening Balance minus Payment Amount minus Transaction Fee

Example: R6 000.00 minus R1 250.00 minus R7.00 = R4 743.00

### Reversal Calculation

If the payment fails after debit:

Final Balance = Temporary Balance plus Payment Amount plus Transaction Fee

Example: R4 743.00 plus R1 250.00 plus R7.00 = R6 000.00

The final balance must return to the original opening balance if the payment did not complete.

## 9. End to End Failed Payment Flow

| Step | Banking Event                       | Expected Result                                   |
|------|-------------------------------------|---------------------------------------------------|
| 1    | Customer logs into banking platform | Login is successful                               |
| 2    | Customer views opening balance      | Balance displays R6 000.00                        |
| 3    | Customer starts payment             | Payment screen opens                              |
| 4    | Customer enters payment details     | Amount and recipient are accepted                 |
| 5    | Customer confirms payment           | Payment request is submitted                      |
| 6    | System temporarily debits account   | Balance becomes R4 743.00                         |
| 7    | Payment service timeout occurs      | Payment fails                                     |
| 8    | System creates reversal             | Debit and fee are restored                        |
| 9    | Final balance updates               | Balance returns to R6 000.00                      |
| 10   | Transaction history is checked      | Failed and reversed records are shown             |
| 11   | Statement is checked                | Statement reflects failure and reversal correctly |
| 12   | Admin portal is checked             | Internal record shows full status journey         |
| 13   | API response is validated           | API returns reversed or failed status             |
| 14   | Database is checked                 | Backend records match expected values             |

## 10. Detailed Test Case 1

### Test Case ID

TC BANK REV 001

### Test Case Title

Verify that a failed payment reverses the debit and restores the customer balance

### Test Objective

To confirm that when a payment fails after the customer account is debited, the system reverses the debit and restores the customer balance correctly.

### Preconditions

| Precondition                   | Description                                        |
|--------------------------------|----------------------------------------------------|
| Customer account exists        | Valid funded account is available                  |
| Account is funded              | Opening balance is R6 000.00                       |
| Recipient account exists       | Recipient account is available for payment attempt |
| Payment feature is enabled     | Customer can initiate a payment                    |
| Failure condition is available | Payment timeout can be simulated                   |
| Transaction logging is enabled | History and admin records can be checked           |
| API access is available        | Payment status can be validated through API        |
| Database access is available   | Backend records can be queried                     |

### Test Steps

| Step | Action | Expected Result |
|------|--------|-----------------|
|------|--------|-----------------|

|    |                                       |                                                    |
|----|---------------------------------------|----------------------------------------------------|
| 1  | Login using customer account CUST3001 | Login is successful                                |
| 2  | Open account dashboard                | Account balance displays R6 000.00                 |
| 3  | Start a payment transaction           | Payment screen opens                               |
| 4  | Enter recipient account ACC4001       | Recipient account is accepted                      |
| 5  | Enter payment amount of R1 250.00     | Amount is accepted                                 |
| 6  | Confirm R7.00 transaction fee         | Fee is displayed before submission                 |
| 7  | Submit payment                        | Payment request is created                         |
| 8  | Simulate payment service timeout      | Payment fails after debit attempt                  |
| 9  | Check temporary account balance       | Balance may temporarily show R4 743.00             |
| 10 | Wait for reversal processing          | Reversal is created successfully                   |
| 11 | Check final account balance           | Balance returns to R6 000.00                       |
| 12 | Open transaction history              | Failed and reversed records are displayed          |
| 13 | Check statement view                  | Statement reflects debit and reversal correctly    |
| 14 | Check admin portal                    | Status journey is visible internally               |
| 15 | Query database records                | Debit, failure and reversal records are accurate   |
| 16 | Validate API response                 | API returns reversed or failed final status        |
| 17 | Capture evidence                      | Screenshots, SQL result and API response are saved |
| 18 | Record test result                    | Test passes if final balance is restored           |

### Expected Result

The payment should not leave the customer financially affected after failure.

| Field                         | Expected Value |
|-------------------------------|----------------|
| Opening balance               | R6 000.00      |
| Payment amount                | R1 250.00      |
| Transaction fee               | R7.00          |
| Temporary balance after debit | R4 743.00      |
| Reversal amount               | R1 257.00      |
| Final balance after reversal  | R6 000.00      |

## 11. Detailed Test Case 2

### Test Case ID

TC BANK REV 002

### Test Case Title

Verify that failed payment status is displayed correctly across all systems

### Test Objective

To confirm that the failed payment does not appear as completed in any customer facing or backend system.

| Step | Action | Expected Result |
|------|--------|-----------------|
|------|--------|-----------------|

|   |                                     |                                            |
|---|-------------------------------------|--------------------------------------------|
| 1 | Complete failed payment scenario    | Payment fails due to timeout               |
| 2 | Open customer transaction history   | Status displays Failed or Reversed         |
| 3 | Open statement view                 | Status and reversal are shown correctly    |
| 4 | Open admin portal                   | Internal status shows full payment journey |
| 5 | Retrieve payment status through API | API does not return Completed              |
| 6 | Query database transaction status   | Database shows failed or reversed status   |
| 7 | Compare statuses                    | All systems show consistent final status   |
| 8 | Capture evidence                    | Status evidence is saved                   |

### Expected Result

The transaction status should not display as Completed if the payment did not successfully reach the recipient.

| Status    | Meaning                                       |
|-----------|-----------------------------------------------|
| Failed    | Payment did not complete and no debit remains |
| Reversed  | Debit was posted but then restored            |
| Cancelled | Payment was stopped before completion         |

## 12. Detailed Test Case 3

### Test Case ID

TC BANK REV 003

### Test Case Title

Verify that duplicate reversal is prevented

### Test Objective

To confirm that the system does not reverse the same failed payment more than once.

| Step | Action                                    | Expected Result                       |
|------|-------------------------------------------|---------------------------------------|
| 1    | Complete failed payment scenario          | Payment fails and reversal is created |
| 2    | Confirm final balance                     | Balance returns to R6 000.00          |
| 3    | Trigger reversal retry for same reference | System checks existing reversal       |
| 4    | Query transaction records                 | Only one reversal record exists       |
| 5    | Check final balance again                 | Balance remains R6 000.00             |
| 6    | Validate API response                     | API prevents duplicate reversal       |
| 7    | Capture evidence                          | SQL and API evidence are saved        |

Only one reversal should be created for the failed payment reference EFT9001. The customer must not receive an additional credit from a duplicate reversal.

## 13. Reconciliation Table

| Field | Expected | Customer History | Statement | Admin Portal | API Response | Database | Result |
|-------|----------|------------------|-----------|--------------|--------------|----------|--------|
|-------|----------|------------------|-----------|--------------|--------------|----------|--------|

|                   |           |           |           |           |           |           |      |
|-------------------|-----------|-----------|-----------|-----------|-----------|-----------|------|
| Payment reference | EFT9001   | EFT9001   | EFT9001   | EFT9001   | EFT9001   | EFT9001   | Pass |
| Opening balance   | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | Pass |
| Payment amount    | R1 250.00 | R1 250.00 | R1 250.00 | R1 250.00 | R1 250.00 | R1 250.00 | Pass |
| Transaction fee   | R7.00     | R7.00     | R7.00     | R7.00     | R7.00     | R7.00     | Pass |
| Temporary balance | R4 743.00 | R4 743.00 | R4 743.00 | R4 743.00 | R4 743.00 | R4 743.00 | Pass |
| Reversal amount   | R1 257.00 | R1 257.00 | R1 257.00 | R1 257.00 | R1 257.00 | R1 257.00 | Pass |
| Final balance     | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | R6 000.00 | Pass |
| Final status      | Reversed  | Reversed  | Reversed  | Reversed  | Reversed  | Reversed  | Pass |

## 14. SQL Validation Examples

### Query 1: Validate failed payment and reversal records

```
SELECT
    transaction_reference,
    account_number,
    transaction_type,
    amount,
    fee_amount,
    transaction_status,
    balance_after_transaction,
    created_at
FROM bank_transactions
WHERE transaction_reference = 'EFT9001'
ORDER BY created_at ASC;
```

| transaction_reference | account_number | type     | amount  | fee_amount | status    | balance_after_transaction |
|-----------------------|----------------|----------|---------|------------|-----------|---------------------------|
| EFT9001               | ACC3001        | DEBIT    | 1250.00 | 7.00       | FAILED    | 4743.00                   |
| EFT9001               | ACC3001        | REVERSAL | 1257.00 | 0.00       | COMPLETED | 6000.00                   |

### Query 2: Validate final account balance

```
SELECT
    customer_id,
    account_number,
    available_balance,
    ledger_balance,
    currency,
    updated_at
FROM bank_accounts
WHERE account_number = 'ACC3001';
```

| customer_id | account_number | available_balance | ledger_balance | currency |
|-------------|----------------|-------------------|----------------|----------|
| CUST3001    | ACC3001        | 6000.00           | 6000.00        | ZAR      |



Query 3: Validate duplicate reversal prevention

```
SELECT
    transaction_reference,
    transaction_type,
    COUNT(*) AS reversal_count
FROM bank_transactions
WHERE transaction_reference = 'EFT9001'
AND transaction_type = 'REVERSAL'
GROUP BY transaction_reference, transaction_type;
```

| transaction_reference | transaction_type | reversal_count |
|-----------------------|------------------|----------------|
| EFT9001               | REVERSAL         | 1              |

15. API Validation Examples

API Test 1: Get Failed Payment Status

Request

```
GET /api/payments/eft/EFT9001
```

Expected Response

```
{
  "transactionReference": "EFT9001",
  "paymentType": "EFT",
  "senderAccount": "ACC3001",
  "recipientAccount": "ACC4001",
  "amount": 1250.00,
  "feeAmount": 7.00,
  "currency": "ZAR",
  "status": "REVERSED",
  "failureReason": "PAYMENT_SERVICE_TIMEOUT",
  "reversalAmount": 1257.00
}
```

| Assertion            | Expected Result         |
|----------------------|-------------------------|
| Status code          | 200 OK                  |
| transactionReference | EFT9001                 |
| paymentType          | EFT                     |
| amount               | 1250.00                 |
| feeAmount            | 7.00                    |
| status               | REVERSED                |
| failureReason        | PAYMENT_SERVICE_TIMEOUT |
| reversalAmount       | 1257.00                 |

API Test 2: Get Account Balance After Reversal

Request

```
GET /api/accounts/ACC3001/balance
```

Expected Response

```
{
  "customerId": "CUST3001",
  "accountNumber": "ACC3001",
  "availableBalance": 6000.00,
  "ledgerBalance": 6000.00,
  "currency": "ZAR",
  "status": "ACTIVE"
}
```

| Assertion        | Expected Result |
|------------------|-----------------|
| Status code      | 200 OK          |
| accountNumber    | ACC3001         |
| availableBalance | 6000.00         |
| ledgerBalance    | 6000.00         |
| currency         | ZAR             |
| status           | ACTIVE          |

16. Negative Test Scenarios

| Test Case ID    | Scenario                                          | Expected Result                                     |
|-----------------|---------------------------------------------------|-----------------------------------------------------|
| TC BANK REV 004 | Payment fails before debit                        | No debit record is posted                           |
| TC BANK REV 005 | Payment fails after debit                         | Reversal is created                                 |
| TC BANK REV 006 | Reversal fails                                    | Defect is logged and transaction remains unresolved |
| TC BANK REV 007 | Duplicate reversal is attempted                   | Duplicate reversal is blocked                       |
| TC BANK REV 008 | Failed payment displays as completed              | Defect is logged                                    |
| TC BANK REV 009 | API shows reversed but database shows completed   | Defect is logged                                    |
| TC BANK REV 010 | Statement does not show reversal                  | Defect is logged                                    |
| TC BANK REV 011 | Customer history shows debit only                 | Defect is logged                                    |
| TC BANK REV 012 | Admin portal does not show failure reason         | Defect is logged                                    |
| TC BANK REV 013 | Final balance does not return to original balance | Critical defect is logged                           |

17. Sample Defect Report

Defect ID

DEF BANK REV 001

Defect Title

Failed payment remains deducted from customer account after reversal process

Severity

Critical

## Priority

High

## Environment

QA Banking Web Build 1.0.6

## Module

Payment Reversal and Recovery

## Related Test Case

TC BANK REV 001

## Description

After a payment fails due to a payment service timeout, the customer account remains debited even though the payment did not complete. No successful reversal is reflected in the customer account balance.

The opening balance was R6 000.00. After a failed payment of R1 250.00 and R7.00 fee, the balance remains R4 743.00 instead of returning to R6 000.00.

## Steps to Reproduce

| Step | Action                                |
|------|---------------------------------------|
| 1    | Login using customer account CUST3001 |
| 2    | Confirm opening balance is R6 000.00  |
| 3    | Start an EFT payment to ACC4001       |
| 4    | Enter payment amount of R1 250.00     |
| 5    | Confirm R7.00 transaction fee         |
| 6    | Submit the payment                    |
| 7    | Simulate payment service timeout      |
| 8    | Wait for reversal processing          |
| 9    | Check customer account balance        |
| 10   | Check transaction history             |
| 11   | Query database records                |
| 12   | Validate payment status through API   |

## Expected Result

The failed payment should be reversed and the customer balance should return to R6 000.00.

| Field           | Expected Value |
|-----------------|----------------|
| Opening balance | R6 000.00      |
| Payment amount  | R1 250.00      |
| Fee amount      | R7.00          |
| Reversal amount | R1 257.00      |
| Final balance   | R6 000.00      |

## Actual Result

The account balance remains R4 743.00. The transaction status shows failed, but the reversal is not reflected in the customer account balance.

## Evidence

| Evidence             | Description                                        |
|----------------------|----------------------------------------------------|
| Screenshot 1         | Opening balance of R6 000.00                       |
| Screenshot 2         | Payment amount and fee confirmation                |
| Screenshot 3         | Payment failure message                            |
| Screenshot 4         | Final balance still showing R4 743.00              |
| Screenshot 5         | Transaction history showing failed payment         |
| SQL result           | No completed reversal record found                 |
| API response         | Payment status returned as failed                  |
| Reconciliation table | Expected final balance differs from actual balance |

## Business Impact

The customer loses access to funds even though the payment did not complete. This may result in customer complaints, financial harm, manual investigation and serious loss of trust in the banking platform.

## Compliance Risk

The system does not restore funds after a failed transaction. This creates a financial integrity and audit risk because the failed payment leaves the account in an incorrect state.

## Recommendation

The reversal workflow should be reviewed to ensure that failed payments are automatically restored when the debit has already been posted. The system should create a completed reversal record, update the customer balance and return a final status that accurately reflects the outcome.

The defect should remain open until the failed payment is retested successfully across the areas below.

| Retest Area         | Expected Result                           |
|---------------------|-------------------------------------------|
| Customer balance    | Restored to R6 000.00                     |
| Transaction history | Shows failed payment and reversal         |
| Statement           | Shows accurate debit and reversal entries |
| Admin portal        | Shows failure reason and reversal status  |
| API response        | Returns correct final status              |
| Database            | Contains completed reversal record        |

## 18. Test Execution Summary

| Metric                | Count |
|-----------------------|-------|
| Total test cases      | 13    |
| Passed                | 9     |
| Failed                | 1     |
| Blocked               | 0     |
| Not run               | 3     |
| Defects logged        | 1     |
| Critical defects      | 1     |
| High priority defects | 1     |

## 19. Summary of Findings

Testing confirmed that the payment failure was detected by the system and that the transaction did not proceed to a successful recipient credit. However, one critical issue was identified where the customer balance remained debited after the failed payment.

The reversal process did not restore the customer funds, leaving the account balance incorrect.

## 20. Final QA Recommendation

Based on the executed failed payment and reversal test scope, the product should not proceed to release until the reversal defect has been corrected and successfully retested.

The corrected build should confirm that when a payment fails after debit, the system automatically restores the full deducted amount, including any transaction fee where applicable.

The final status and balance should match across the following sources.

| Source                       |
|------------------------------|
| Customer account dashboard   |
| Customer transaction history |
| Statement view               |
| Admin portal                 |
| API response                 |
| Database record              |

## 21. Skills Demonstrated

| Skill                   | How It Was Demonstrated                                       |
|-------------------------|---------------------------------------------------------------|
| Functional testing      | Tested failed payment flow from initiation to reversal        |
| Financial validation    | Verified debit, fee, reversal and final balance values        |
| Negative testing        | Tested failure scenarios rather than only successful payments |
| Recovery testing        | Validated system behaviour after payment timeout              |
| SQL validation          | Queried backend payment and reversal records                  |
| API testing             | Validated payment status and balance responses                |
| Reconciliation testing  | Compared dashboard, statement, admin, API and database        |
| Defect reporting        | Logged a critical defect with steps and evidence              |
| Risk analysis           | Explained customer, business and compliance impact            |
| Banking domain thinking | Applied transaction recovery logic to digital banking         |

## 22. Portfolio Disclaimer

All examples in this case study are fictional and created for portfolio purposes. No real banking system, customer account, employer system, client system or confidential data is used. The case study is designed to demonstrate transferable QA skills from regulated transactional testing into banking and FinTech quality assurance.